

RAPPORT DE PROJET DE FIN D'ETUDES

Conception et développement d'une plateforme web de gestion du parc TPE
bancaire

Présenté par : **Nessim Ayadi**

Organisme d'accueil : Bank ABC Tunisie

Encadrant académique : Kmar Fersi

Encadrant professionnel : Mr. Alaa Laamouri

Année universitaire : 2025-2026

Remerciements

Je tiens à exprimer ma profonde gratitude à Bank ABC Tunisie pour m'avoir accueilli dans un environnement professionnel stimulant et formateur. Ce stage de projet de fin d'études m'a permis de confronter les connaissances acquises durant mon cursus à des problématiques bancaires concrètes, exigeantes et fortement orientées qualité de service.

Je remercie mon encadrante académique Kmar Fersi pour son accompagnement, ses conseils méthodologiques et son suivi pédagogique. Je remercie également mon encadrant professionnel Mr. Alaa Laamouri pour sa disponibilité, ses orientations et les échanges qui ont permis de mieux comprendre les besoins du métier monétique.

Mes remerciements s'adressent aussi aux membres de l'équipe qui ont facilité l'accès aux informations nécessaires, validé les choix fonctionnels et contribué à l'amélioration progressive de la solution. Enfin, je remercie ma famille et mes proches pour leur soutien tout au long de cette période.

Table des matières

Remerciements	2
Table des matières	3
Liste des abréviations	7
Liste des figures	8
Liste des tableaux	8
Introduction Générale	9
Chapitre 1 - Contexte général et étude préliminaire	10
1.1. Présentation de l'organisme d'accueil	10
1.1.1. La Banque ABC : profil et positionnement	10
1.1.2. La Direction Monétique	11
1.1.3. Contexte du stage	11
1.2. Etude de l'existant et problématique	11
1.2.1. Analyse des processus existants	12
1.2.2. Problématique	12
1.2.3. Etude comparative des solutions existantes	13
1.3. Objectifs du projet	13
1.4. Méthodologie de développement	14
1.4.1. Choix de la méthodologie	14
1.4.2. Organisation des sprints	15
1.4.3. Environnement de développement	15
Chapitre 2 - Analyse et spécification des besoins	17
2.1. Identification des acteurs	17
2.1.1. Acteurs principaux	17
2.2. Besoins fonctionnels	18

2.2.1. Gestion des terminaux TPE	18
2.2.2. Gestion des commerçants	19
2.2.3. Gestion des demandes d'attribution	19
2.2.4. Gestion des pannes et maintenance	20
2.2.5. Gestion des taux de commission - Règle des quatre yeux	20
2.3. Besoins non fonctionnels	21
2.4. Diagrammes de cas d'utilisation	21
2.4.1. Diagramme global	22
2.4.2. Spécification des cas d'utilisation principaux	23
2.5. Matrice des permissions	23
Chapitre 3 - Conception générale et technique	25
3.1. Architecture générale du système	25
3.1.1. Architecture en couches	26
3.1.2. Architecture de sécurité	26
3.2. Modèle de données	27
3.2.1. Entités principales	28
3.2.2. Dictionnaire de données	28
3.3. Diagrammes de séquence	29
3.3.1. Séquence - Workflow de demande TPE	30
3.3.2. Séquence - Règle des quatre yeux	30
3.4. Architecture technique détaillée	31
3.4.1. Stack technologique backend	32
3.4.2. Stack technologique frontend	32
3.4.3. Base de données - SQL Server	33
3.5. Algorithme de génération du TID (Luhn)	33
Chapitre 4 - Réalisation, tests et validation	35

4.1. Module d'authentification et de sécurité	35
4.1.1. Implémentation de la sécurité JWT	35
4.1.2. Interface de connexion	36
4.2. Tableau de bord principal	36
4.3. Module de gestion des TPE	37
4.3.1. Liste et consultation des TPE	37
4.3.2. Formulaire de création d'un TPE	37
4.4. Module de gestion des commerçants	38
4.5. Module de gestion des demandes	38
4.5.1. Création et suivi des demandes	39
4.5.2. Interface d'affectation	39
4.6. Module de gestion des pannes	40
4.7. Module de gestion des taux	40
4.8. Intégration Power BI et reporting	41
4.9. Module d'import/export Excel	41
4.10. Gestion des notifications	42
4.11. Module d'audit et traçabilité	42
4.12. Tests et validation	42
4.12.1. Tests unitaires et d'intégration	43
4.12.2. Tests fonctionnels	43
Conclusion Générale	45
Bibliographie et Netographie	46
Annexes	47
Annexe A - Endpoints API REST principaux	47
Annexe B - Enumérations du système	51
Annexe C - Structure du projet GitHub	52

Annexe D - Inventaire technique du projet	53
Fiche de validation - Authentification	54
Fiche de validation - Gestion TPE	55
Fiche de validation - Demandes	56
Fiche de validation - Pannes	57
Fiche de validation - Taux	58
Fiche de validation - Reporting	59
Fiche de validation - Permissions	60
Fiche de validation - Import bancaire	61
Fiche pédagogique - Cas d'utilisation créer une demande	62
Fiche pédagogique - Cas d'utilisation valider une demande	63
Fiche pédagogique - Cas d'utilisation affecter un TPE	64
Fiche pédagogique - Cas d'utilisation déclarer une panne	65
Fiche pédagogique - Cas d'utilisation traiter une panne	66
Fiche pédagogique - Cas d'utilisation saisir un taux	67
Fiche pédagogique - Cas d'utilisation valider un taux	68
Fiche pédagogique - Critères d'acceptation sécurité	69
Fiche pédagogique - Critères d'acceptation ergonomie	70
Fiche pédagogique - Critères d'acceptation performance	71
Fiche pédagogique - Difficultés rencontrées	72
Fiche pédagogique - Apports personnels	73
Fiche pédagogique - Risques et mesures	74
Fiche pédagogique - Préparation de la soutenance	75

Dans la version Word, la table des matières est modifiable et peut être mise à jour automatiquement après toute modification du contenu.

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
DTO	Data Transfer Object
JWT	JSON Web Token
KPI	Key Performance Indicator
RBAC	Role-Based Access Control
SQL	Structured Query Language
TID	Terminal Identifier
TPE	Terminal de Paiement Electronique
UI	User Interface
UML	Unified Modeling Language

Liste des figures

Figure	Intitulé
Fig. 1	Architecture logique de la solution
Fig. 2	Architecture physique de déploiement
Fig. 3	Diagramme de cas d'utilisation global
Fig. 4	Diagramme de composants
Fig. 5	Diagramme de classes métier
Fig. 6	Diagramme de séquence - demande, validation et affectation
Fig. 7	Workflow de validation 4 yeux
Fig. 8	Méthodologie Agile Scrum

Liste des tableaux

Tableau	Intitulé
Tab. 1	Inventaire technique du projet
Tab. 2	Acteurs et responsabilités
Tab. 3	Besoins fonctionnels
Tab. 4	Besoins non fonctionnels
Tab. 5	Matrice des permissions
Tab. 6	Dictionnaire de données simplifié
Tab. 7	Plan de tests

Introduction Générale

Cette partie traite de le contexte général, la problématique et la démarche globale du PFE. Elle présente les éléments utiles pour comprendre le rôle de cette section dans le projet et pour relier les choix effectués aux besoins de la banque. La rédaction reste centrée sur le travail réalisé durant le stage, conformément aux consignes pédagogiques du rapport de fin d'études.

L'objectif n'est pas seulement de décrire le contexte général, la problématique et la démarche globale du PFE, mais d'expliquer la démarche suivie, les contraintes rencontrées et les décisions prises pour aboutir à une solution exploitable. Le projet s'inscrit dans un environnement bancaire où la traçabilité, la sécurité, la qualité des données et la continuité de service sont des exigences fortes.

Le présent rapport décrit la conception et le développement d'une plateforme web destinée à digitaliser la gestion du parc TPE bancaire. L'application couvre les demandes d'attribution, la gestion des terminaux, les commerçants, les affectations, les pannes, les taux de commission et les indicateurs de pilotage.

La problématique principale réside dans la transformation de processus manuels, dispersés et difficilement traçables en un système centralisé, sécurisé et exploitable par plusieurs profils. Le projet apporte une réponse à cette problématique à travers une architecture full-stack, une gestion fine des rôles et des workflows métier adaptés au contexte bancaire.

Chapitre 1 - Contexte général et étude préliminaire

Ce chapitre présente l'organisme d'accueil, le contexte de stage, l'étude de l'existant, la problématique, les objectifs et la méthodologie adoptée. Il sert de base à la compréhension des besoins qui seront détaillés dans les chapitres suivants.

1.1. Présentation de l'organisme d'accueil

Dans le cadre de 1.1. Présentation de l'organisme d'accueil, le travail a consisté à analyser l'environnement institutionnel et métier de Bank ABC Tunisie en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement profil bancaire, organisation interne, service monétique, contexte du stage. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

1.1.1. La Banque ABC : profil et positionnement

Dans le cadre de 1.1.1. La Banque ABC : profil et positionnement, le travail a consisté à analyser le positionnement de la banque et son rôle dans les services financiers en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement services aux particuliers, solutions entreprises, moyens de paiement, banque digitale. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité

applicative, validation et documentation.

1.1.2. La Direction Monétique

Dans le cadre de 1.1.2. La Direction Monétique, le travail a consisté à analyser les missions liées aux moyens de paiement et aux terminaux TPE en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement gestion des terminaux, suivi commerçants, contrôle des taux, qualité de service. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

1.1.3. Contexte du stage

Dans le cadre de 1.1.3. Contexte du stage, le travail a consisté à analyser la mission confiée durant le projet de fin d'études en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement digitalisation, analyse de l'existant, développement full-stack, validation fonctionnelle. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

1.2. Etude de l'existant et problématique

Dans le cadre de 1.2. Etude de l'existant et problématique, le travail a consisté à analyser les processus manuels de gestion TPE et leurs limites en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement données dispersées, délais de traitement, risques d'erreurs, absence de KPI. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

1.2.1. Analyse des processus existants

Dans le cadre de 1.2.1. Analyse des processus existants, le travail a consisté à analyser le cycle actuel de demande, validation, affectation et maintenance en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement demande agence, validation monétique, stock TPE, traitement des pannes. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

1.2.2. Problématique

Dans le cadre de 1.2.2. Problématique, le travail a consisté à analyser la question centrale de digitalisation et de traçabilité en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement centralisation, sécurité, automatisation, pilotage temps réel. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

1.2.3. Etude comparative des solutions existantes

Dans le cadre de 1.2.3. Etude comparative des solutions existantes, le travail a consisté à analyser les limites des outils bureautiques et des solutions non intégrées en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Excel, Access, applications isolées, progiciels lourds. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

1.3. Objectifs du projet

Dans le cadre de 1.3. Objectifs du projet, le travail a consisté à analyser les objectifs fonctionnels, techniques et pédagogiques en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement réduire les erreurs, sécuriser les accès, automatiser TID, fournir dashboards. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

1.4. Méthodologie de développement

Dans le cadre de 1.4. Méthodologie de développement, le travail a consisté à analyser l'organisation Agile Scrum retenue en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement sprints, backlog, revues, amélioration continue. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

1.4.1. Choix de la méthodologie

Dans le cadre de 1.4.1. Choix de la méthodologie, le travail a consisté à analyser la justification de l'approche Agile en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement évolutivité des besoins, feedback métier, priorisation, livraison incrémentale. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

1.4.2. Organisation des sprints

Dans le cadre de 1.4.2. Organisation des sprints, le travail a consisté à analyser la planification des modules dans le temps en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement socle technique, TPE, demandes, pannes, reporting. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Sprint	Objectif principal	Livrables
Sprint 1	Cadrage et socle technique	Architecture, sécurité initiale, modèle de données
Sprint 2	Gestion TPE et commerçants	CRUD, statuts, recherche, import
Sprint 3	Demandes et affectations	Workflow agence-monétique, validation et mise en service
Sprint 4	Pannes, taux et 4 yeux	Maintenance, contrôle des taux, audit
Sprint 5	Pilotage et finalisation	Dashboards, Power BI, tests, documentation

1.4.3. Environnement de développement

Dans le cadre de 1.4.3. Environnement de développement, le travail a consisté à analyser les outils et technologies utilisés pendant la réalisation en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Java 17, Spring Boot, Angular, SQL Server, Git. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

En conclusion, ce premier chapitre a permis de consolider les bases nécessaires pour la suite du rapport. Les éléments présentés démontrent que la solution n'est pas uniquement une application de gestion, mais un système cohérent qui répond à une problématique de digitalisation, de contrôle et de pilotage du parc TPE.

Chapitre 2 - Analyse et spécification des besoins

Ce chapitre traduit la problématique en besoins. Il identifie les acteurs, décrit les exigences fonctionnelles et non fonctionnelles, présente les cas d'utilisation et formalise les permissions associées aux profils.

2.1. Identification des acteurs

Dans le cadre de 2.1. Identification des acteurs, le travail a consisté à analyser les profils qui interagissent avec la plateforme en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Agence, Monétique, Inputer, Authorizer, Admin. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Acteur	Responsabilités principales
Agence	Créer des demandes, suivre leur état, déclarer des pannes, consulter les informations autorisées
Monétique	Gérer le stock TPE, valider les demandes, affecter les terminaux, suivre les incidents
Inputer	Saisir les données monétiques et les taux en attente de validation
Authorizer	Valider ou rejeter les saisies sensibles selon la règle des quatre yeux
Admin	Administrer les utilisateurs, les rôles, les écrans et les permissions

2.1.1. Acteurs principaux

Dans le cadre de 2.1.1. Acteurs principaux, le travail a consisté à analyser les responsabilités de chaque utilisateur en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement création demande, validation, affectation, administration. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

2.2. Besoins fonctionnels

Dans le cadre de 2.2. Besoins fonctionnels, le travail a consisté à analyser les fonctions attendues par le système en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement gestion TPE, gestion commerçants, demandes, pannes, taux. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Besoin	Description	Module implémenté
Gestion TPE	Création, modification, statut, import, TID	TPEController, TPEService
Commerçants	Informations commerciales et bancaires	CommercantController, CommercantService
Demandes	Workflow de demande et validation	DemandeController, DemandeService
Pannes	Déclaration, diagnostic, réparation et test	PanneController, PanneService
Taux	Saisie, soumission, validation 4 yeux	TauxController, TauxService
Reporting	KPI, tableaux de bord, Power BI	DashboardController, PowerBIController

2.2.1. Gestion des terminaux TPE

Dans le cadre de 2.2.1. Gestion des terminaux TPE, le travail a consisté à analyser le suivi du stock et du cycle de vie des terminaux en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement numéro de série, TID, statut, historique. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

2.2.2. Gestion des commerçants

Dans le cadre de 2.2.2. Gestion des commerçants, le travail a consisté à analyser la gestion des informations légales et opérationnelles des commerçants en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement raison sociale, compte, activité, e-commerce. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

2.2.3. Gestion des demandes d'attribution

Dans le cadre de 2.2.3. Gestion des demandes d'attribution, le travail a consisté à analyser le workflow entre agence et monétique en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement création, saisie monétique, validation, affectation. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

2.2.4. Gestion des pannes et maintenance

Dans le cadre de 2.2.4. Gestion des pannes et maintenance, le travail a consisté à analyser le suivi des incidents et interventions en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement déclaration, diagnostic, réparation, test. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

2.2.5. Gestion des taux de commission - Règle des quatre yeux

Dans le cadre de 2.2.5. Gestion des taux de commission - Règle des quatre yeux, le travail a consisté à analyser le contrôle séparé entre saisie et validation en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Inputer, Authorizer, motif rejet, audit. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de

réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

2.3. Besoins non fonctionnels

Dans le cadre de 2.3. Besoins non fonctionnels, le travail a consisté à analyser les critères de qualité attendus en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement sécurité, maintenabilité, performance, traçabilité. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Critère	Exigence	Réponse technique
Sécurité	Accès contrôlé et authentifié	JWT, Spring Security, RBAC
Traçabilité	Historique des actions sensibles	AuditService, AuditLog
Maintenabilité	Code organisé par couches	Controller, Service, Repository, DTO
Performance	Pagination et recherche	Spring Data Pageable, dashboards dédiés
Ergonomie	Interfaces adaptées aux rôles	Angular, guards, sidebar dynamique

2.4. Diagrammes de cas d'utilisation

Dans le cadre de 2.4. Diagrammes de cas d'utilisation, le travail a consisté à analyser la modélisation des interactions utilisateur en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement acteurs, cas d'usage, limites système, droits. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment

la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

2.4.1. Diagramme global

Dans le cadre de 2.4.1. Diagramme global, le travail a consisté à analyser la vue synthétique des fonctions accessibles en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Agence, Monétique, Admin, 4 yeux. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

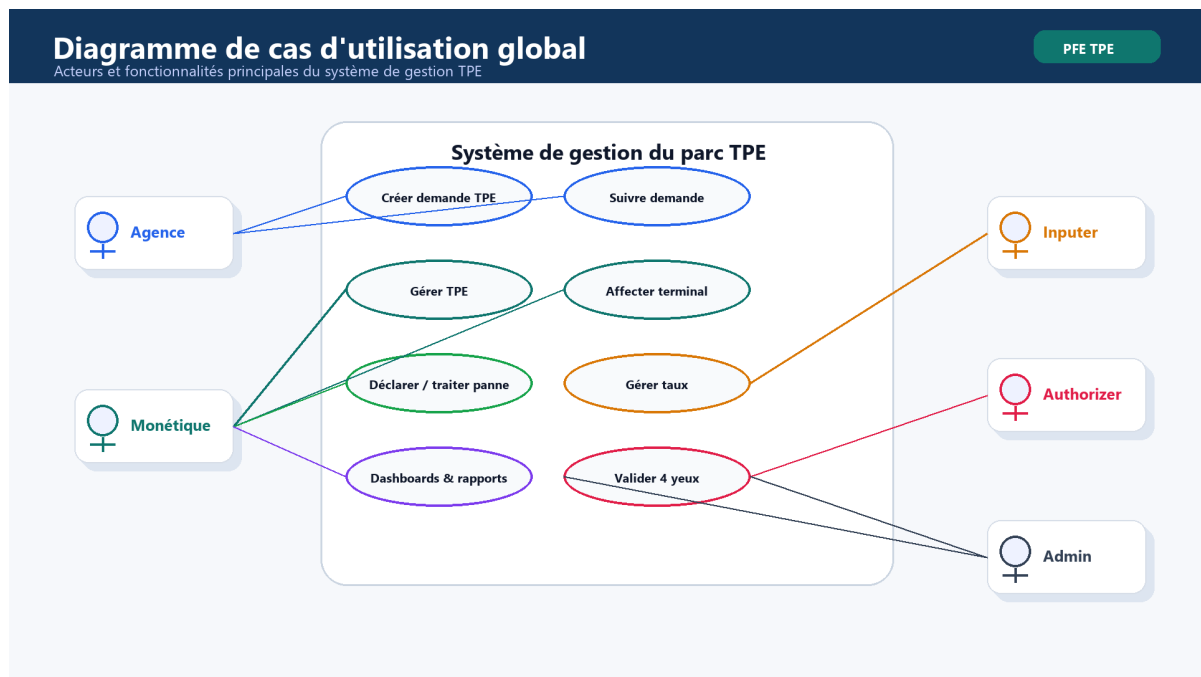


Fig. 3 - Diagramme de cas d'utilisation global

2.4.2. Spécification des cas d'utilisation principaux

Dans le cadre de 2.4.2. Spécification des cas d'utilisation principaux, le travail a consisté à analyser la description détaillée des scénarios majeurs en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement préconditions, scénario nominal, exceptions, résultat. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

2.5. Matrice des permissions

Dans le cadre de 2.5. Matrice des permissions, le travail a consisté à analyser l'association entre rôles, écrans et actions en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement RBAC, permissions écran, sécurité API, guards Angular. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Action	Agence	Monétique	Inputer	Authorizer	Admin
Créer demande	Oui	Non	Non	Non	Oui
Valider saisie demande	Non	Oui	Oui	Oui	Oui
Créer TPE	Non	Oui	Non	Non	Oui
Affecter TPE	Non	Oui	Non	Non	Oui
Saisir taux	Non	Non	Oui	Non	Oui
Valider taux	Non	Non	Non	Oui	Oui
Administrer permissions	Non	Non	Non	Non	Oui

En conclusion, ce deuxième chapitre a permis de consolider les bases nécessaires pour la suite du rapport. Les éléments présentés démontrent que la solution n'est pas uniquement une application de gestion, mais un système cohérent qui répond à une problématique de digitalisation, de contrôle et de pilotage du parc TPE.

Chapitre 3 - Conception générale et technique

Ce chapitre présente la conception retenue. Il détaille l'architecture générale, les choix de sécurité, le modèle de données, les diagrammes de séquence, la stack technique et l'algorithme de génération du numéro terminal.

3.1. Architecture générale du système

Dans le cadre de 3.1. Architecture générale du système, le travail a consisté à analyser la structure full-stack de la solution en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Angular, Spring Boot, services métier, SQL Server. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.



Fig. 1 - Architecture logique de la solution

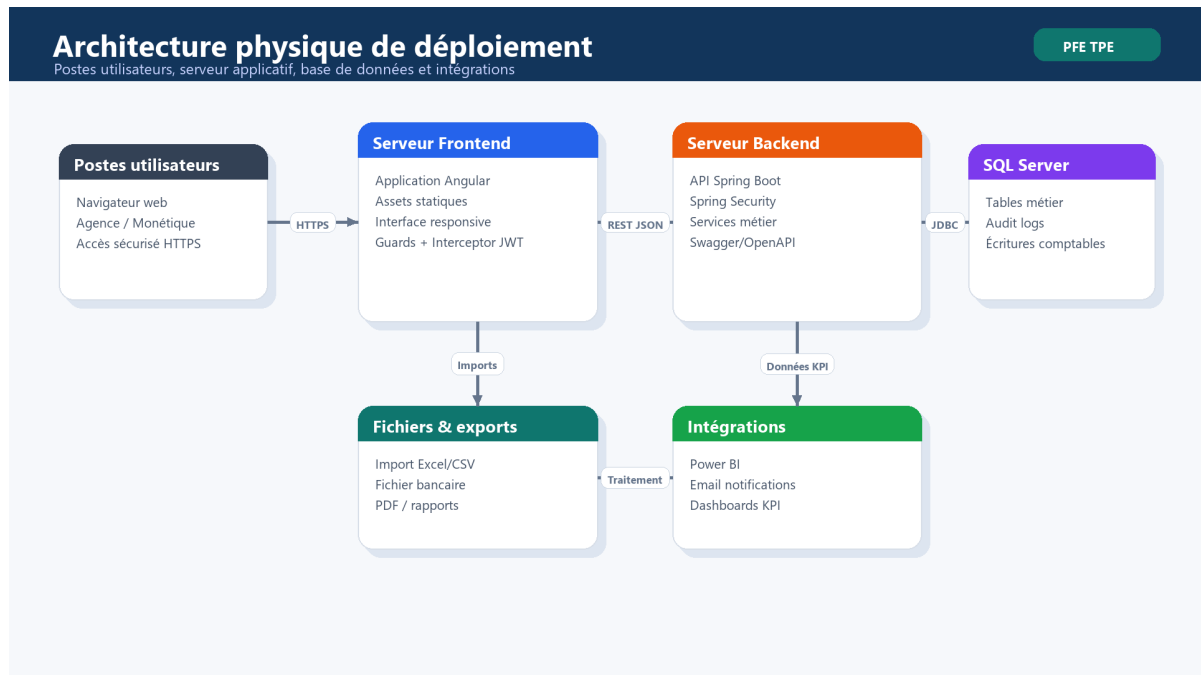


Fig. 2 - Architecture physique de déploiement

3.1.1. Architecture en couches

Dans le cadre de 3.1.1. Architecture en couches, le travail a consisté à analyser la séparation entre présentation, application, domaine et persistance en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement UI, API, service, repository. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

3.1.2. Architecture de sécurité

Dans le cadre de 3.1.2. Architecture de sécurité, le travail a consisté à analyser la protection des ressources et des workflows sensibles en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement JWT, RBAC, PreAuthorize, AuthGuard. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

3.2. Modèle de données

Dans le cadre de 3.2. Modèle de données, le travail a consisté à analyser les entités métier et leurs relations en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement User, TPE, Commerçant, Demande, Affectation, Panne, Taux. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

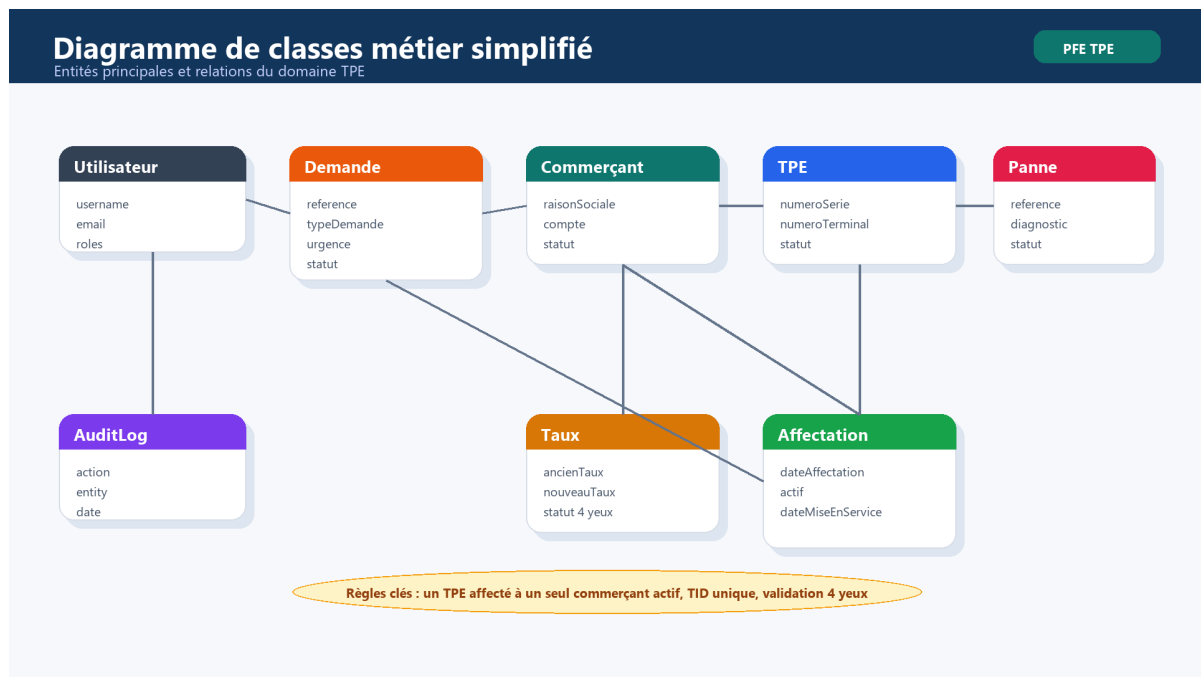


Fig. 5 - Diagramme de classes métier simplifié

3.2.1. Entités principales

Dans le cadre de 3.2.1. Entités principales, le travail a consisté à analyser les objets persistants manipulés par la plateforme en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement BaseEntity, statuts, relations JPA, audit. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

3.2.2. Dictionnaire de données

Dans le cadre de 3.2.2. Dictionnaire de données, le travail a consisté à analyser les attributs essentiels des tables métier en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement identifiants, champs métier, statuts, dates. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Entité	Attributs clés	Rôle
TPE	numeroSerie, numeroTerminal, typeTPE, statut	Représente un terminal physique ou e-commerce
Commerçant	raisonSociale, numeroCompte, codeAgence, statut	Regroupe les informations du commerçant
Demande	reference, urgence, statut, commentaireValidation	Pilote le workflow agence-monétique
Affectation	dateAffectation, actif, dateMiseEnService	Lie un TPE à un commerçant
Panne	reference, diagnostic, actionCorrective, statut	Suit l'incident technique
Taux	ancienTaux, nouveauTaux, inputer, autorizer, statut	Applique la règle des quatre yeux

3.3. Diagrammes de séquence

Dans le cadre de 3.3. Diagrammes de séquence, le travail a consisté à analyser les scénarios dynamiques les plus importants en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement demande, validation, affectation, taux. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité

applicative, validation et documentation.

3.3.1. Séquence - Workflow de demande TPE

Dans le cadre de 3.3.1. Séquence - Workflow de demande TPE, le travail a consisté à analyser l'enchaînement de création, validation et affectation en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Frontend, API, Service, Repository. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

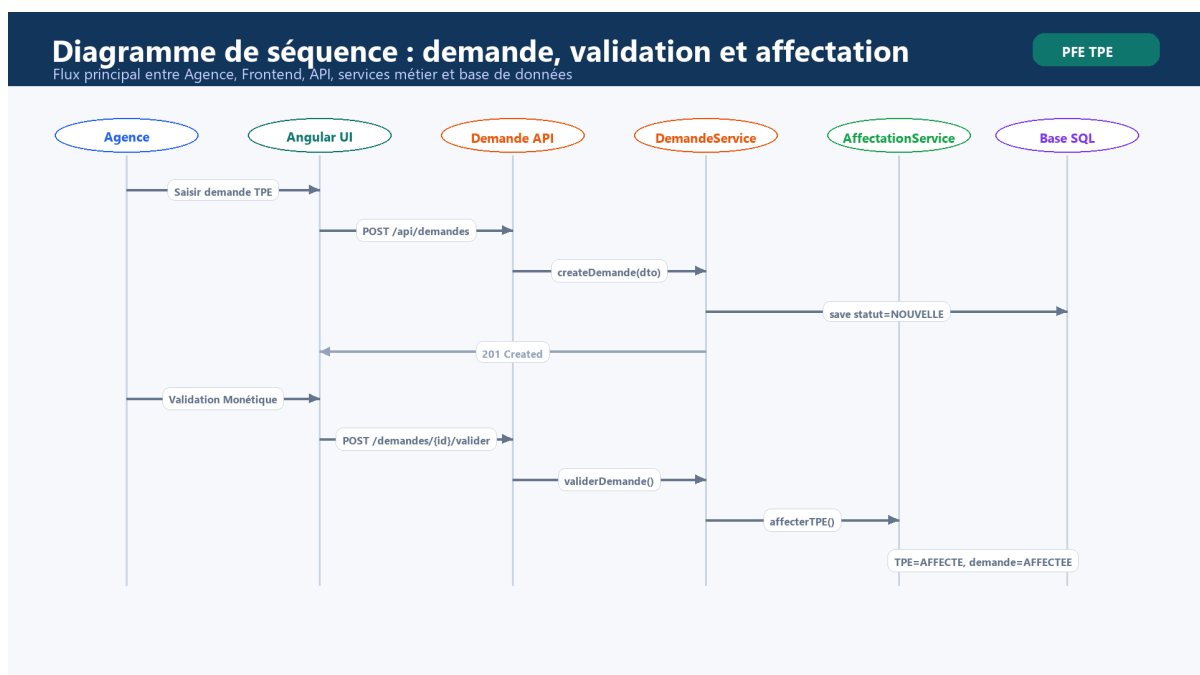


Fig. 6 - Diagramme de séquence de validation et affectation

3.3.2. Séquence - Règle des quatre yeux

Dans le cadre de 3.3.2. Séquence - Règle des quatre yeux, le travail a consisté à analyser le contrôle entre saisie et validation en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Inputer, Authorizer, blocage auto-validation, audit. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

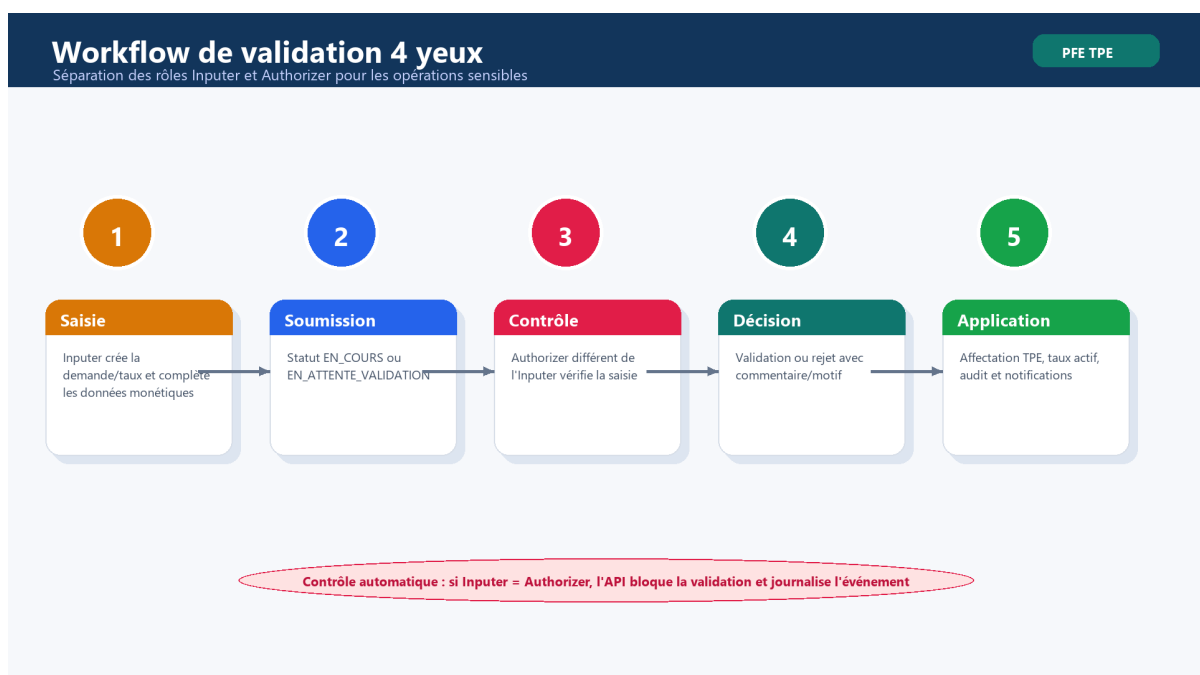


Fig. 7 - Workflow de validation 4 yeux

3.4. Architecture technique détaillée

Dans le cadre de 3.4. Architecture technique détaillée, le travail a consisté à analyser les choix technologiques et leurs rôles en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement backend, frontend, base de données, intégrations. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux

utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

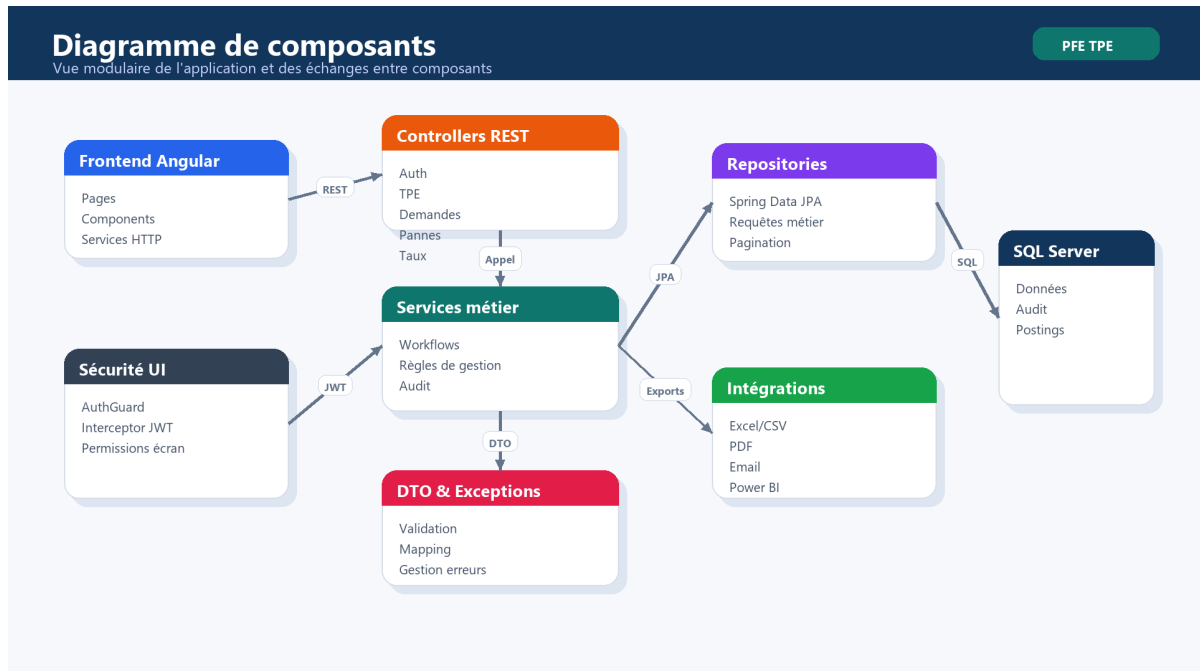


Fig. 4 - Diagramme de composants

3.4.1. Stack technologique backend

Dans le cadre de 3.4.1. Stack technologique backend, le travail a consisté à analyser les dépendances Java utilisées en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Spring Boot 3.2.1, Java 17, JPA, OpenAPI, Apache POI. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

3.4.2. Stack technologique frontend

Dans le cadre de 3.4.2. Stack technologique frontend, le travail a consisté à analyser les dépendances Angular utilisées en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Angular 14, RxJS, Bootstrap, Chart.js, Power BI client. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

3.4.3. Base de données - SQL Server

Dans le cadre de 3.4.3. Base de données - SQL Server, le travail a consisté à analyser le stockage relationnel et les migrations en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement tables, relations, index, scripts SQL. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

3.5. Algorithme de génération du TID (Luhn)

Dans le cadre de 3.5. Algorithme de génération du TID (Luhn), le travail a consisté à analyser la génération et la validation du numéro terminal en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement RIB, code agence, compteur, clé Luhn. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Etape	Description
1	Extraire les deux premiers chiffres du RIB
2	Ajouter le code agence sur trois chiffres
3	Ajouter un compteur terminal sur trois chiffres
4	Calculer la clé de contrôle avec l'algorithme de Luhn
5	Vérifier l'unicité du numéro terminal en base

Exemple pédagogique : pour un RIB commençant par 23, une agence 041 et un compteur 008, la base du TID devient 23041008. La clé calculée par Luhn complète le numéro terminal et permet de détecter certaines erreurs de saisie.

En conclusion, ce troisième chapitre a permis de consolider les bases nécessaires pour la suite du rapport. Les éléments présentés démontrent que la solution n'est pas uniquement une application de gestion, mais un système cohérent qui répond à une problématique de digitalisation, de contrôle et de pilotage du parc TPE.

Chapitre 4 - Réalisation, tests et validation

Ce chapitre présente le travail développé. Il décrit les modules livrés, les interfaces principales, les intégrations, les mécanismes d'audit et les tests réalisés pour valider la solution.

4.1. Module d'authentification et de sécurité

Dans le cadre de 4.1. Module d'authentification et de sécurité, le travail a consisté à analyser l'accès sécurisé à l'application en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement login, JWT, rôles, permissions. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.1.1. Implémentation de la sécurité JWT

Dans le cadre de 4.1.1. Implémentation de la sécurité JWT, le travail a consisté à analyser la génération, validation et propagation du token en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement JwtTokenProvider, JwtAuthenticationFilter, SecurityConfig. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.1.2. Interface de connexion

Dans le cadre de 4.1.2. Interface de connexion, le travail a consisté à analyser l'écran d'authentification Angular en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement LoginComponent, AuthService, interceptor, guard. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.2. Tableau de bord principal

Dans le cadre de 4.2. Tableau de bord principal, le travail a consisté à analyser le pilotage global par indicateurs en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement total TPE, pannes, demandes, affectations. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Indicateur	Description	Source
Total TPE	Nombre de terminaux en stock	TPERepository
TPE disponibles	Terminaux prêts à être affectés	StatutTPE.DISPONIBLE
Pannes en cours	Incidents non résolus	PanneRepository
Affectations actives	Terminaux liés aux commerçants	AffectationRepository

Indicateur	Description	Source
Taux de disponibilité	Disponibles et affectés sur total parc	DashboardService

4.3. Module de gestion des TPE

Dans le cadre de 4.3. Module de gestion des TPE, le travail a consisté à analyser la création et le suivi des terminaux en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement liste, formulaire, statut, import. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.3.1. Liste et consultation des TPE

Dans le cadre de 4.3.1. Liste et consultation des TPE, le travail a consisté à analyser la recherche et la pagination des terminaux en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement TpeListComponent, TpeService, GET /tpes. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.3.2. Formulaire de création d'un TPE

Dans le cadre de 4.3.2. Formulaire de création d'un TPE, le travail a consisté à analyser la saisie d'un terminal physique ou e-commerce en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement TpeFormComponent, validation, TID. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.4. Module de gestion des commerçants

Dans le cadre de 4.4. Module de gestion des commerçants, le travail a consisté à analyser la gestion des informations commerçants en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement MerchantList, MerchantForm, compte, e-commerce. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.5. Module de gestion des demandes

Dans le cadre de 4.5. Module de gestion des demandes, le travail a consisté à analyser le workflow d'attribution entre agence et monétique en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement DemandForm, DemandList, validation. Cette analyse a permis de transformer un besoin métier exprimé de

manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.5.1. Création et suivi des demandes

Dans le cadre de 4.5.1. Création et suivi des demandes, le travail a consisté à analyser le parcours de la demande depuis l'agence en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement statuts, pièces jointes, commentaires. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.5.2. Interface d'affectation

Dans le cadre de 4.5.2. Interface d'affectation, le travail a consisté à analyser la sélection ou génération du TPE affecté en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement AffectationService, TPE disponible, date mise en service. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.6. Module de gestion des pannes

Dans le cadre de 4.6. Module de gestion des pannes, le travail a consisté à analyser le suivi des incidents et réparations en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement déclaration, diagnostic, test, remplacement. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.7. Module de gestion des taux

Dans le cadre de 4.7. Module de gestion des taux, le travail a consisté à analyser la saisie et validation des taux en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Inputer, Authorizer, soumission, validation. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.8. Intégration Power BI et reporting

Dans le cadre de 4.8. Intégration Power BI et reporting, le travail a consisté à analyser l'exploitation des données pour le pilotage en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement token, reports, dashboards, KPI. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.9. Module d'import/export Excel

Dans le cadre de 4.9. Module d'import/export Excel, le travail a consisté à analyser le chargement massif des données TPE en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Apache POI, TPEImportRecord, export. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.10. Gestion des notifications

Dans le cadre de 4.10. Gestion des notifications, le travail a consisté à analyser l'information des acteurs à chaque étape importante en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement NotificationService, nouvelle demande, validation, affectation. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.11. Module d'audit et traçabilité

Dans le cadre de 4.11. Module d'audit et traçabilité, le travail a consisté à analyser l'enregistrement des actions sensibles en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement AuditService, AuditLog, succès, erreur. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.12. Tests et validation

Dans le cadre de 4.12. Tests et validation, le travail a consisté à analyser les vérifications fonctionnelles et techniques en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement tests unitaires, tests API, tests workflow, Postman. Cette analyse a permis de transformer un besoin métier exprimé de manière

opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Type de test	Objectif	Exemple
Unitaire	Valider une règle isolée	TauxServiceTest
Intégration API	Contrôler endpoints et sécurité	Postman collection
Fonctionnel	Valider scénario utilisateur	Création demande puis affectation
Sécurité	Contrôler accès par rôle	PreAuthorize et AuthGuard
Non-régression	Vérifier après correction	Build Maven et Angular

4.12.1. Tests unitaires et d'intégration

Dans le cadre de 4.12.1. Tests unitaires et d'intégration, le travail a consisté à analyser la validation des services critiques en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement TauxServiceTest, règle 4 yeux, transactions. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

4.12.2. Tests fonctionnels

Dans le cadre de 4.12.2. Tests fonctionnels, le travail a consisté à analyser la validation des scénarios utilisateurs en tenant compte des usages réels observés dans le domaine monétique. Les

points étudiés concernent principalement login, demande, affectation, panne, taux. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

En conclusion, ce quatrième chapitre a permis de consolider les bases nécessaires pour la suite du rapport. Les éléments présentés démontrent que la solution n'est pas uniquement une application de gestion, mais un système cohérent qui répond à une problématique de digitalisation, de contrôle et de pilotage du parc TPE.

Conclusion Générale

Le projet de fin d'études a permis de concevoir et de développer une plateforme web complète pour la gestion du parc TPE bancaire. La solution répond à une problématique réelle de centralisation, de digitalisation, de traçabilité et de pilotage des activités monétiques.

Les principaux objectifs ont été atteints : gestion des TPE, gestion des commerçants, workflow de demandes, affectation, suivi des pannes, gestion des taux avec règle des quatre yeux, tableaux de bord, import/export et contrôle d'accès par rôles. Le projet s'appuie sur une architecture en couches claire, avec Spring Boot pour le backend, Angular pour le frontend et SQL Server pour la persistance.

Sur le plan pédagogique, ce travail a permis de renforcer les compétences en analyse métier, conception UML, développement full-stack, sécurité applicative, modélisation de données, tests et documentation. Il a également montré l'importance de la communication avec les acteurs métier pour transformer une problématique bancaire en solution exploitable.

Les perspectives d'évolution concernent l'application mobile pour les techniciens, les notifications temps réel, l'automatisation de la facturation, l'espace self-service commerçant, l'analyse prédictive des pannes et la mise en place d'une chaîne DevOps plus complète.

Bibliographie et Netographie

Guide pédagogique du rapport de stage / projet de fin d'études fourni comme référence.

Documentation officielle Spring Boot - <https://spring.io/projects/spring-boot>

Documentation officielle Spring Security - <https://spring.io/projects/spring-security>

Documentation Angular - <https://angular.dev>

Documentation Microsoft SQL Server - <https://learn.microsoft.com/sql/sql-server>

Documentation OpenAPI / Swagger - <https://swagger.io/specification>

Documentation JSON Web Token - <https://jwt.io/introduction>

Apache POI - <https://poi.apache.org>

Microsoft Power BI Embedded Analytics -
<https://learn.microsoft.com/power-bi/developer/embedded/>

Annexes

Annexe A - Endpoints API REST principaux

Cette annexe reprend les principaux endpoints exposés par le backend. La liste permet de comprendre la couverture fonctionnelle de l'API et le niveau de sécurisation appliqué aux opérations.

Contrôleur	Méthode	Chemin	Sécurité
AffectationController	POST	/api/affectations	Public / contrôlé ailleurs
AffectationController	GET	/api/affectations	"hasAnyRole('ADMIN', 'MONETIQUE')"
AffectationController	GET	/api/affectations/{id}	"hasAnyRole('ADMIN', 'MONETIQUE', 'AGENCE')"
AffectationController	GET	/api/affectations/commercant/{commercantId}	"hasAnyRole('ADMIN', 'MONETIQUE', 'AGENCE')"
AffectationController	GET	/api/affectations/actives	"hasAnyRole('ADMIN', 'MONETIQUE', 'AGENCE')"
AffectationController	POST	/api/affectations/{id}/mise-en-service	"hasAnyRole('ADMIN', 'MONETIQUE', 'AGENCE')"
AffectationController	POST	/api/affectations/{id}/desaffecter	"hasAnyRole('ADMIN', 'MONETIQUE')"
AuthController	POST	/api/auth/login	Public / contrôlé ailleurs
AuthController	POST	/api/auth/register	Public / contrôlé ailleurs
CommercantController	POST	/api/commercents, /api/commercant	Public / contrôlé ailleurs
CommercantController	GET	/api/commercents, /api/commercant/{id}	"hasAnyRole('MONETIQUE', 'AGENCE', 'ADMIN')"
CommercantController	GET	/api/commercents, /api/commercant	"hasAnyRole('MONETIQUE', 'AGENCE', 'ADMIN')"
CommercantController	GET	/api/commercents, /api/commercant/search	"hasAnyRole('MONETIQUE', 'AGENCE', 'ADMIN')"
CommercantController	GET	/api/commercents, /api/commercant/agence/{codeAgence}	"hasAnyRole('MONETIQUE', 'AGENCE', 'ADMIN')"
CommercantController	PUT	/api/commercents, /api/commercant/{id}	"hasAnyRole('MONETIQUE', 'AGENCE', 'ADMIN')"
CommercantController	PATCH	/api/commercents, /api/commercant/{id}/statut	"hasAnyRole('MONETIQUE', 'AGENCE', 'ADMIN')"
CommercantController	DELETE	/api/commercents, /api/commercant/{id}	"hasAnyRole('MONETIQUE', 'ADMIN')"
DashboardController	GET	/api/dashboard/stats	Public / contrôlé ailleurs
DashboardController	GET	/api/dashboard/demandes-statut	"hasAnyRole('ADMIN', 'MONETIQUE')"
DashboardController	GET	/api/dashboard/pannes-type	"hasAnyRole('ADMIN', 'MONETIQUE')"
DashboardController	GET	/api/dashboard/evolution-mensuelle	"hasAnyRole('ADMIN', 'MONETIQUE')"

Contrôleur	Méthode	Chemin	Sécurité
DashboardController	GET	/api/dashboard /repartition-statut	"hasAnyRole('ADMIN', 'MONETIQUE')"
DashboardController	GET	/api/dashboard /repartition-type	"hasAnyRole('ADMIN', 'MONETIQUE')"
DashboardController	GET	/api/dashboard /evolution-tpe	"hasAnyRole('ADMIN', 'MONETIQUE')"
DashboardController	GET	/api/dashboard /stats-par-agence	"hasAnyRole('ADMIN', 'MONETIQUE')"
DashboardController	GET	/api/dashboard /pannes-periode	"hasAnyRole('ADMIN', 'MONETIQUE', 'AGENCE')"
DashboardController	GET	/api/dashboard /performance-demandes	"hasAnyRole('ADMIN', 'MONETIQUE')"
DashboardController	GET	/api/dashboard /taux-utilisation	"hasAnyRole('ADMIN', 'MONETIQUE')"
DashboardController	GET	/api/dashboard /top-pannes	"hasAnyRole('ADMIN', 'MONETIQUE')"
DashboardController	GET	/api/dashboard /heatmap-pannes	"hasAnyRole('ADMIN', 'MONETIQUE')"
DashboardController	GET	/api/dashboard /stats-commerçants	"hasAnyRole('ADMIN', 'MONETIQUE')"
DemandeController	POST	/api/demandes, /api/demande	Public / contrôlé ailleurs
DemandeController	GET	/api/demandes, /api/demande /{id}	"hasAnyRole('AGENCE', 'ADMIN')"
DemandeController	GET	/api/demandes, /api/demande	"hasAnyRole('MONETIQUE', 'AGENCE', 'ADMIN')"
DemandeController	PUT	/api/demandes, /api/demande /{id}	"hasAnyRole('MONETIQUE', 'AGENCE', 'ADMIN')"
DemandeController	POST	/api/demandes, /api/demande /{id}/valider	"hasAnyRole('MONETIQUE', 'AGENCE', 'ADMIN')"
DemandeController	POST	/api/demandes, /api/demande /{id}/rejeter	"hasAnyRole('INPUTER', 'AUTHORIZER', 'ADMIN')"
DemandeController	PATCH	/api/demandes, /api/demande /{id}/cloturer	"hasAnyRole('AUTHORIZER', 'ADMIN')"
DemandeController	POST	/api/demandes, /api/demande /{id}/piece-jointe	"hasAnyRole('MONETIQUE', 'ADMIN')"
DemandeController	GET	/api/demandes, /api/demande /{id}/piece-jointe/{fileName}	"hasAnyRole('AGENCE', 'MONETIQUE', 'ADMIN')"
FichierBancaireController	POST	/api/fichier-bancaire /upload	Public / contrôlé ailleurs
FichierBancaireController	GET	/api/fichier-bancaire /stats/{sessionDate}	Public / contrôlé ailleurs
FichierBancaireController	GET	/api/fichier-bancaire /transactions/{sessionDate}	Public / contrôlé ailleurs
FichierBancaireController	GET	/api/fichier-bancaire /test	Public / contrôlé ailleurs
FichierBancaireController	GET	/api/fichier-bancaire /rapport/pdf/{sessionDate}	Public / contrôlé ailleurs
FichierBancaireController	GET	/api/fichier-bancaire /rapport/text/{sessionDate}	Public / contrôlé ailleurs
FileUploadController	POST	/api/file-upload /process	Public / contrôlé ailleurs

Contrôleur	Méthode	Chemin	Sécurité
PanneController	GET	/api/pannes	Public / contrôlé ailleurs
PanneController	GET	/api/pannes/{id}	"hasAnyRole('ADMIN', 'MONETIQUE', 'AGENCE')"
PanneController	GET	/api/pannes/statut/{statut}	"hasAnyRole('ADMIN', 'MONETIQUE', 'AGENCE')"
PanneController	GET	/api/pannes/tpe/{tpeId}	"hasAnyRole('ADMIN', 'MONETIQUE')"
PanneController	GET	/api/pannes/technicien/{technicienId}	"hasAnyRole('ADMIN', 'MONETIQUE', 'AGENCE')"
PanneController	POST	/api/pannes	"hasAnyRole('ADMIN', 'MONETIQUE')"
PanneController	PUT	/api/pannes/{id}	"hasAnyRole('ADMIN', 'MONETIQUE', 'AGENCE')"
PanneController	PUT	/api/pannes/{id}/statut/{statut}	"hasAnyRole('ADMIN', 'MONETIQUE')"
PanneController	POST	/api/pannes/{panneId}/assigner/{technicienId}	"hasAnyRole('ADMIN', 'MONETIQUE')"
PanneController	POST	/api/pannes/{id}/diagnostiquer	"hasAnyRole('ADMIN', 'MONETIQUE')"
PanneController	POST	/api/pannes/{id}/en-reparation	"hasAnyRole('ADMIN', 'MONETIQUE')"
PanneController	POST	/api/pannes/{id}/reparee	"hasAnyRole('ADMIN', 'MONETIQUE')"
PanneController	POST	/api/pannes/{id}/resoudre	"hasAnyRole('ADMIN', 'MONETIQUE')"
PanneController	POST	/api/pannes/{id}/tester	"hasAnyRole('ADMIN', 'MONETIQUE')"
PanneController	POST	/api/pannes/{panneId}/tpe-replacement/{tpeRemplacementId}	"hasAnyRole('ADMIN', 'MONETIQUE')"
PanneController	GET	/api/pannes/periode	"hasAnyRole('ADMIN', 'MONETIQUE')"
PanneController	DELETE	/api/pannes/{id}	"hasAnyRole('ADMIN', 'MONETIQUE')"
PowerBIController	GET	/api/powerbi/token/{reportId}	Public / contrôlé ailleurs
PowerBIController	GET	/api/powerbi/reports	Public / contrôlé ailleurs
PowerBIController	GET	/api/powerbi/reports/{reportId}	Public / contrôlé ailleurs
PowerBIController	GET	/api/powerbi/status	Public / contrôlé ailleurs
RoleController	GET	/api/roles	Public / contrôlé ailleurs
ScreenController	GET	/api/screens	Public / contrôlé ailleurs
ScreenController	GET	/api/screens/active	"hasAnyRole('ADMIN', 'MONETIQUE')"
ScreenController	GET	/api/screens/{id}	Public / contrôlé ailleurs
ScreenController	GET	/api/screens/code/{code}	"hasAnyRole('ADMIN', 'MONETIQUE')"

Contrôleur	Méthode	Chemin	Sécurité
ScreenController	GET	/api/screens /user/{username}	"hasAnyRole('ADMIN', 'MONETIQUE')"
ScreenController	GET	/api/screens /me	Public / contrôlé ailleurs
ScreenController	GET	/api/screens /permissions/{screenCode}	Public / contrôlé ailleurs
ScreenController	POST	/api/screens	Public / contrôlé ailleurs
ScreenController	PUT	/api/screens /{id}	"hasRole('ADMIN')"
ScreenController	DELETE	/api/screens /{id}	"hasRole('ADMIN')"

Annexe B - Enumérations du système

Enumération	Valeurs
RoleType	ROLE_MONETIQUE, ROLE_AGENCE, ROLE_INPUTER, ROLE_AUTHORIZER, ROLE_ADMIN
StatutTPE	DISPONIBLE, RESERVE, AFFECTE, EN_PANNE, MAINTENANCE, HORS_SERVICE
StatutDemande	NOUVELLE, EN_COURS, VALIDEE_MONETIQUE, AFFECTEE, CLOTUREE, REJETEE
StatutPanne	DECLAREE, DIAGNOSTIQUEE, EN_REPARATION, REPAREE, TESTEE, IRRECUPERABLE
StatutTaux	BROUILLON, EN_ATTENTE_VALIDATION, VALIDE, REJETE
TypeTPE	PHYSIQUE, ECOMMERCE
Urgence	BASSE, NORMALE, HAUTE, CRITIQUE selon le contexte métier

Annexe C - Structure du projet GitHub

L'application est organisée en deux parties principales. Le dossier TPE contient le backend Spring Boot, tandis que le dossier front end contient l'application Angular. La séparation facilite le développement, le déploiement et la maintenance.

Zone	Contenu	Rôle
TPE/src/main/java/.../controller	14 contrôleurs	Exposition des endpoints REST
TPE/src/main/java/.../service	16 services	Implémentation des règles métier
TPE/src/main/java/.../entity	18 entités	Modèle persistant JPA
TPE/src/main/java/.../repository	18 repositories	Accès aux données
front end/src/app	34 composants Angular	Interfaces utilisateur
front end/src/app/models	10 modèles TypeScript	Contrats côté frontend

Annexe D - Inventaire technique du projet

Indicateur	Valeur
Contrôleurs backend	14
Services backend	16
Entités JPA	18
Repositories	18
Composants Angular	34
Modèles TypeScript	10
Diagrammes design générés	8

Cet inventaire donne une vue synthétique du volume de réalisation. Il permet également de justifier que le rapport décrit un travail réalisé et non une simple proposition théorique.

Fiche de validation - Authentification

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Connexion avec identifiants valides	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Rejet d'un mot de passe incorrect	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Stockage et injection du token JWT	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Redirection après expiration	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche de validation - Authentification, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Connexion avec identifiants valides, Rejet d'un mot de passe incorrect, Stockage et injection du token JWT, Redirection après expiration. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche de validation - Gestion TPE

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Création d'un terminal	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Recherche et pagination	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Changement de statut	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Génération du TID	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche de validation - Gestion TPE, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Création d'un terminal, Recherche et pagination, Changement de statut, Génération du TID. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche de validation - Demandes

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Création par agence	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Saisie monétique	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Validation Authorizer	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Affectation automatique ou manuelle	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche de validation - Demandes, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Création par agence, Saisie monétique, Validation Authorizer, Affectation automatique ou manuelle. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche de validation - Pannes

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Déclaration	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Diagnostic	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Réparation	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Test et remise en disponibilité	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche de validation - Pannes, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Déclaration, Diagnostic, Réparation, Test et remise en disponibilité. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche de validation - Taux

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Création par Inputer	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Soumission	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Blocage auto-validation	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Validation par Authorizer	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche de validation - Taux, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Création par Inputer, Soumission, Blocage auto-validation, Validation par Authorizer. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche de validation - Reporting

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Chargement des KPI	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Répartition par statut	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Evolution mensuelle	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Export	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche de validation - Reporting, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Chargement des KPI, Répartition par statut, Evolution mensuelle, Export. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche de validation - Permissions

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Accès par rôle	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Masquage des écrans	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Contrôle backend	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Journalisation	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche de validation - Permissions, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Accès par rôle, Masquage des écrans, Contrôle backend, Journalisation. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche de validation - Import bancaire

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Lecture fichier	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Vérification TPE	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Génération écritures	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Rapport de session	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche de validation - Import bancaire, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Lecture fichier, Vérification TPE, Génération écritures, Rapport de session. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Cas d'utilisation créer une demande

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Acteur Agence identifié	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Préconditions vérifiées	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Données commerçant saisies	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Statut initial NOUVELLE	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Cas d'utilisation créer une demande, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Acteur Agence identifié, Préconditions vérifiées, Données commerçant saisies, Statut initial NOUVELLE. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Cas d'utilisation valider une demande

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Acteur Inputer ou Authorizer	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Règle métier appliquée	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Transitions EN_COURS et VALIDEE_MONETIQUE	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Notification envoyée	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Cas d'utilisation valider une demande, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Acteur Inputer ou Authorizer, Règle métier appliquée, Transitions EN_COURS et VALIDEE_MONETIQUE, Notification envoyée. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Cas d'utilisation affecter un TPE

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
TPE disponible	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Commerçant actif	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Affectation active créée	Conforme si l'écran, l'API et la base reflètent le comportement prévu
TPE marqué AFFECTE	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Cas d'utilisation affecter un TPE, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement TPE disponible, Commerçant actif, Affectation active créée, TPE marqué AFFECTE. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Cas d'utilisation déclarer une panne

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
TPE existant	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Panne référencée	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Statut TPE EN_PANNE	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Suivi technique disponible	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Cas d'utilisation déclarer une panne, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement TPE existant, Panne référencée, Statut TPE EN_PANNE, Suivi technique disponible. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Cas d'utilisation traiter une panne

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Diagnostic renseigné	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Réparation suivie	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Résultat de test	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Retour disponible ou hors service	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Cas d'utilisation traiter une panne, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Diagnostic renseigné, Réparation suivie, Résultat de test, Retour disponible ou hors service. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Cas d'utilisation saisir un taux

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Utilisateur Inputer	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Ancien taux récupéré	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Nouveau taux enregistré	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Statut BROUILLON	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Cas d'utilisation saisir un taux, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Utilisateur Inputer, Ancien taux récupéré, Nouveau taux enregistré, Statut BROUILLON. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Cas d'utilisation valider un taux

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Utilisateur Authorizer	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Inputer différent	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Motif obligatoire en rejet	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Activation du taux validé	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Cas d'utilisation valider un taux, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Utilisateur Authorizer, Inputer différent, Motif obligatoire en rejet, Activation du taux validé. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Critères d'acceptation sécurité

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Token obligatoire	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Endpoints protégés	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Rôles vérifiés côté serveur	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Ecrans filtrés côté client	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Critères d'acceptation sécurité, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Token obligatoire, Endpoints protégés, Rôles vérifiés côté serveur, Ecrans filtrés côté client. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Critères d'acceptation ergonomie

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Navigation claire	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Messages d'erreur compréhensibles	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Formulaires guidés	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Actions visibles selon rôle	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Critères d'acceptation ergonomie, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Navigation claire, Messages d'erreur compréhensibles, Formulaires guidés, Actions visibles selon rôle. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Critères d'acceptation performance

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Pagination des listes	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Chargement ciblé des KPI	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Imports suivis	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Aucun blocage critique observé	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Critères d'acceptation performance, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Pagination des listes, Chargement ciblé des KPI, Imports suivis, Aucun blocage critique observé. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Difficultés rencontrées

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Alignement frontend-backend	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Gestion des statuts	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Règle des quatre yeux	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Données de test cohérentes	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Difficultés rencontrées, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Alignement frontend-backend, Gestion des statuts, Règle des quatre yeux, Données de test cohérentes. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Apports personnels

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Compréhension du métier monétique	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Conception UML	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Sécurité applicative	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Validation et documentation	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Apports personnels, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Compréhension du métier monétique, Conception UML, Sécurité applicative, Validation et documentation. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Risques et mesures

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Erreur de saisie	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Validation non autorisée	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Perte de traçabilité	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Indisponibilité du stock	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Risques et mesures, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Erreur de saisie, Validation non autorisée, Perte de traçabilité, Indisponibilité du stock. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.

Fiche pédagogique - Préparation de la soutenance

Cette fiche est ajoutée en annexe pour faciliter la relecture pédagogique et la préparation de la soutenance. Elle synthétise les scénarios à présenter, les résultats attendus et les preuves à montrer pendant la démonstration.

Point de contrôle	Résultat attendu
Démonstration courte	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Scénario métier fluide	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Diagrammes lisibles	Conforme si l'écran, l'API et la base reflètent le comportement prévu
Résultats et limites assumés	Conforme si l'écran, l'API et la base reflètent le comportement prévu

Dans le cadre de Fiche pédagogique - Préparation de la soutenance, le travail a consisté à analyser la validation fonctionnelle du module en tenant compte des usages réels observés dans le domaine monétique. Les points étudiés concernent principalement Démonstration courte, Scénario métier fluide, Diagrammes lisibles, Résultats et limites assumés. Cette analyse a permis de transformer un besoin métier exprimé de manière opérationnelle en exigences fonctionnelles, techniques et pédagogiquement justifiables.

La solution retenue privilégie une architecture claire, séparant la présentation Angular, l'API REST Spring Boot, les services métier, les repositories JPA et la base SQL Server. Cette séparation rend le système plus maintenable et permet de localiser les règles critiques, notamment la règle des quatre yeux, la génération du TID et les transitions de statuts.

Le rôle joué durant le projet a donc été double. D'une part, il a fallu comprendre les processus existants et leurs limites. D'autre part, il a fallu concevoir et développer une application capable de réduire les traitements manuels, d'améliorer la fiabilité des informations et de fournir aux utilisateurs des écrans adaptés à leurs responsabilités.

Sur le plan pédagogique, cette partie montre la capacité à passer d'une problématique bancaire concrète à une réalisation logicielle structurée. Elle met en évidence les compétences mobilisées en analyse, conception, modélisation, développement backend, développement frontend, sécurité applicative, validation et documentation.